

CONTAINER MANAGEMENT



HELLO FLASK

app.py

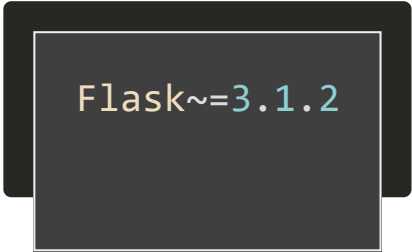
```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "<h1>Ciao! Sono un'app in un container!</h1>"

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=8080)
```



requirements.txt



```
Flask~=3.1.2
```



DOCKERIZZAZIONE LOCALE

Dockerfile

```
# Immagine Base
FROM python:3.11-slim

# Directory di lavoro
WORKDIR /app

# Dipendenze
COPY requirements.txt .
RUN pip install -r requirements.txt

# Codice Sorgente
COPY . .

# Comando di avvio
CMD ["python", "app.py"]
```



BUILD LOCALE

1. Build dell'immagine:

Crea un pacchetto con tutto il necessario (OS, Python, App).

```
docker build -t app-saluto .
```



RUN LOCALE

2. Esecuzione (Run):

```
docker run -p 8080:8080 app-saluto
```

L'app è ora accessibile su <http://localhost:8080>.



AGGIUNGERE CONTAINER (REDIS)

Vogliamo contare quante volte la pagina viene visitata. Un semplice container Python non basta perché se si riavvia, perde la memoria. Aggiungiamo un database: Redis.

Nuova Architettura:

- **Container 1 (Web):** L'app Flask (logica).
- **Container 2 (DB):** Redis (memoria dati).



REFACTORING APP.PY

```
import redis
from flask import Flask

# Connessione al servizio chiamato "redis"
cache = redis.Redis(host='redis', port=6379)

app = Flask(__name__)

@app.route('/')
def home():
    # Incrementa il contatore nel DB
    count = cache.incr('hits')
    return f"<h1>Questa pagina è stata vista {count} volte.</h1>"

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=8080)
```



ORCHESTRAZIONE CON DOCKER COMPOSE

Gestire due container manualmente (avviarli, collegarli in rete) è difficile.

`docker-compose.yml`:

```
version: '3'
services:
  web:
    build: .
    ports: ["8080:8080"]
    depends_on: [redis]

  redis:
    image: "redis:alpine"
```



ESECUZIONE DELL'ORCHESTRAZIONE

```
docker-compose up --build
```

Cosa succede "dietro le quinte":

1. Viene creata una rete **bridge** interna.
2. Il servizio **web** può vedere il servizio **redis** tramite DNS interno (hostname **redis**).
3. L'app è pronta e persistente.



AVANZATE



SHELL

A volte i log non bastano. Bisogna entrare nel container mentre è in esecuzione.
Aprire una Shell:

```
# Sintassi: docker exec -it <container_name_or_id> <shell>
docker exec -it my-flask-app /bin/bash
# Oppure 'sh' se 'bash' non è installato (es. Alpine Linux)
docker exec -it my-redis-db sh
```



INSPECT

Per vedere metadati --- IP, variabili d'ambiente, volumi montati, configurazioni di rete...

```
docker inspect my-flask-app
```



PERSISTENZA E VOLUMI

- **Volume:** Area di storage gestita da Docker (isolata dall'host). Ideale per Database.
- **Bind Mount:** Collega una cartella del tuo computer (Host) a una cartella nel container. Ideale per **Sviluppo** e **Live Reload**.



ESEMPIO: BIND MOUNT PER HOT-RELOAD

Vogliamo che se modifichiamo il codice `app.py` sul nostro PC, il container si aggiorni subito senza dover ricostruire l'immagine.

Modifica al `docker-compose.yml`:

```
services:
  web:
    build: .
    # Mappa la cartella corrente (.) in /app nel container
    volumes:
      - ./app
    environment:
      - FLASK_DEBUG=1 # Attiva il "watchdog" di Flask
```

Con questa configurazione, Flask "guarda" il disco. Se cambi un file, il server si ☐avvia da solo.

AMMINISTRAZIONE DA CLI

Gestione Container:

```
docker ps          # Lista container attivi
docker ps -a       # Lista TUTTI i container (anche fermi)
docker rm <id>      # Rimuove un container fermo
docker rm -f <id>   # Forza la rimozione (anche se attivo)
```



Gestione Immagini:

```
docker images      # Lista immagini scaricate  
docker rmi <id>    # Rimuove un'immagine
```



Pulizia Totale (Nuclear Option):

```
docker system prune -a --volumes
```

Attenzione: Cancella tutto ciò che non è in uso!



esercitazione



ESTENSIONE

- backend pokeapi
- aggiunta di cache locale
- frontend pokedex



EXTRA/FAQ



DOCKER E PYCHARM

PyCharm **Professional** Edition offre un'integrazione nativa che permette di utilizzare un container Docker come interprete Python, garantendo che l'ambiente di esecuzione (dependencies, OS, ecc.) sia sempre identico a quello di produzione.

1. Assicurati che il tuo progetto abbia un **Dockerfile** che definisca chiaramente l'ambiente (come l'esempio del tuo progetto precedente).
2. Per dire a PyCharm di usare Docker, segui questi passi:
 1. Vai su **File** (o **PyCharm** su macOS) -> **Settings** (o **Preferences**).
 2. Cerca la sezione **Project**: -> **Python Interpreter**.
 3. Clicca sull'icona a forma di ingranaggio e seleziona **Add....** Nella finestra **Add Python Interpreter**, seleziona: **On Docker** (per usare un singolo Dockerfile).



